

Алгоритмы оптимизации

Градиентные методы обучения

После того, как вы “построили” loss-функцию, следующий шаг – найти параметры, которые бы позволили минимизировать ее. Это проблема оптимизации. Чтобы ее решить, часто используется оптимизационный алгоритм – **метод градиентного спуска**.

Градиент спуск – это лыжник, который находится на склоне снежной горы. Снежная гора – это и есть loss-функция.

Существует много разновидностей метода градиентного спуска, семейство этих методов называют **градиентные методы обучения**. Общая идея таких методов: в какой-то точке рассчитать направление наискорейшего спуска, затем использовать данное направление, чтобы прийти в новую точку с меньшим loss. Точка в данном случае обозначает набор параметров.

Постановка задачи

Рассмотрим задачу минимизации функции потерь $J : \cdot \rightarrow \mathbb{R}$, которая переводит произвольный вход в скалярный выход. В машинном обучении J это данные для обучения и параметры θ . Зафиксируем данные для обучения, тогда наша цель решить задачу:

$$\theta^* = \arg \min_{\theta} J(\theta) \quad (1)$$

Многие оптимизаторы работают итеративно, обновляя параметры. При обновлении оптимизатор может оценивать вид функции потерь. Оптимизаторы отличаются алгоритмами обновления параметров.

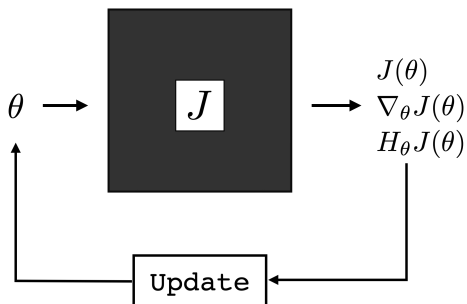


Рис. 1 – Общий алгоритм оптимизации

Градиентные методы различных порядков

Простейший случай – оптимизация **нулевого** порядка, когда обновляющая функция имеет информацию только о значениях $J(\theta)$. В таком случае, единственный способ минимизировать $J(\theta)$, это найти параметры θ перебором и выбрать такие, при которых $J(\theta)$ будет уменьшаться.

В градиентных методах **первого** порядка обновляющая функция рассчитывает градиент функции потерь в текущей точке, т.е. $\nabla_{\theta} J(\theta)$. Это раскрывает информацию о том, в каком направлении необходимо двигаться, чтобы быстрее прийти к минимуму.

В градиентных методах более **высокого** порядка вычисляются производные высоких порядков, например, гессиан H , который описывает кривизну функции в данной точке. Расчет гессиана достаточно ресурсозатратен, но во многих методах рассчитывается его аппроксимация, и такие методы набирают популярность.

Базовый метод GD (Gradient Descent)

Algorithm 10.1: Gradient descent (GD)

- 1 **Input:** objective function J , initial parameter vector θ^0 , learning rate η , number of steps K
 - 2 **Output:** trained parameter vector $\theta^* = \theta^K$
 - 3 **for** $k=0, \dots, K-1$ **do**
 - 4 $\theta^{k+1} \leftarrow \theta^k - \eta \nabla_{\theta} J(\theta^k)$
-

Рис. 2 – Оптимизация функции $J : \theta \rightarrow \mathbb{R}$ с помощью градиента $\nabla_{\theta} J$.

В этом алгоритме есть два гиперпараметра: шаг обучения η (learning rate), который задает размер шага, и количество шагов K . Если η достаточно мал и начальная догадка θ^0 случайна, то алгоритм сойдется к локальному минимуму J за $K \rightarrow \infty$. Чтобы увеличить скорость поиска, можно увеличить шаг обучения.

Планировщик скорости обучения

Learning Rate Schedules. Хорошая практика – начинать с большого шага обучения η и затем уменьшать. Существует много алгоритмов изменения η , их можно описать через вызов некоторой функции $lr(\eta^0, k)$ на каждой итерации:

$$\eta^k = lr(\eta^0, k). \quad (2)$$

Общая концепция: необходимо правило обновления, когда $\eta^{k+1} < \eta^k$. Некоторые популярные подходы:

1. $lr(\eta^0, k) = \beta^{-k} \eta^0$ – **экспоненциальный спад**;
2. $lr(\eta^0, k) = \beta^{-k/M} \eta^0$ – **ступенчато-экспоненциальный спад**;
3. $lr(\eta^0, k) = \frac{K-k}{K} \eta^0$ – **линейный спад**;

Параметры β и M – дополнительные гиперпараметры этого метода.

Algorithm 10.2: GD+lr-schedule.

```
1 Input: objective function  $J$ , initial parameter vector  $\theta^0$ , initial learning rate  $\eta^0$ ,  
   learning rate function  $\text{lr}$ , number of steps  $K$   
2 Output: trained parameter vector  $\theta^* = \theta^K$   
3 for  $k = 0, \dots, K - 1$  do  
4    $\eta^k \leftarrow \text{lr}(\eta^0, k)$   
5    $\theta^{k+1} \leftarrow \theta^k - \eta^k \nabla_{\theta} J(\theta^k)$ 
```

Рис. 3 – Градиентный спуск+планировщик шага

Шаг может меняться, когда достигается плато loss-функции при обучении, или по более сложному нелинейному закону, например, по косинусу.

Момент (momentum)

Можно ли сделать более умное обновление весов, чем просто двигаться в направлении градиента? Можно использовать знание с предыдущего шага, или так называемый, момент: обновление происходит в направлении $\mathbf{v}^{k+1}$, которое является взвешенной суммой предыдущего направления $\mathbf{v}^k$ и отрицательного градиента:

$$\mathbf{v}^{k+1} = \mu \mathbf{v}^k - \eta \nabla_{\theta} J(\theta^k). \quad (3)$$

Вес μ – новый гиперпараметр, который называют моментом (momentum).

Градиентный спуск с моментом.

Algorithm 10.3: GD+momentum.

```
1 Input: objective function  $J$ , initial parameter vector  $\theta^0$ , learning rate  $\eta$ , momentum  
    $\mu$ , number of steps  $K$   
2 Output: trained parameter vector  $\theta^* = \theta^K$   
3  $\mathbf{v}^0 = \mathbf{0}$   
4 for  $k=0, \dots, K-1$  do  
5    $\mathbf{v}^{k+1} = \mu \mathbf{v}^k - \eta \nabla_{\theta} J(\theta^k)$   
6    $\theta^{k+1} \leftarrow \theta^k + \mathbf{v}^{k+1}$ 
```

Градиентный спуск с моментом

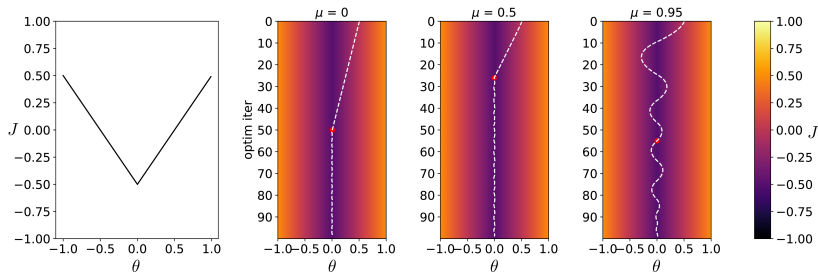


Рис. 4 – Простая loss-функция $J = \text{abs}(\theta)$ (слева). Справа: траектории оптимизации при трех разных значениях μ . Белая линия равна параметру θ на каждой итерации, начиная сверху и заканчивая внизу. Цветом обозначено значение Loss. Красная точка показывает θ , где loss впервые достиг оптимального значения 0.01.

Мини-пакетный градиентный спуск I

Считать градиенты для каждой точки в наборе данных может быть очень ресурсозатратно, особенно если этих данных много (например, 1e6, тогда придется столько раз считать производные). Поэтому вместо обычного градиентного спуска придумали реализовать mini-batch gradient descent, в котором градиенты считаются на выбранном подмножестве данных. Перепишем:

$$\nabla_{\theta} J(\theta, \mathbf{x}^{(i)}, \mathbf{y}^{(i)}) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \mathcal{L}(f_{\theta}(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})), \quad (4)$$

Для больших N считать дорого. Выберем случайно подмножество данных из N : $\mathbf{x}^{(b)}, \mathbf{y}^{(b)}$, где B – размер батча. Тогда посчитаем полный градиент как средний градиент на этом батче:

$$\tilde{\mathbf{g}} = \frac{1}{B} \sum_{b=1}^B \nabla_{\theta} \mathcal{L}(f_{\theta}(\mathbf{x}^{(b)}, \mathbf{y}^{(b)})) \quad (5)$$

Мини-пакетный градиентный спуск II

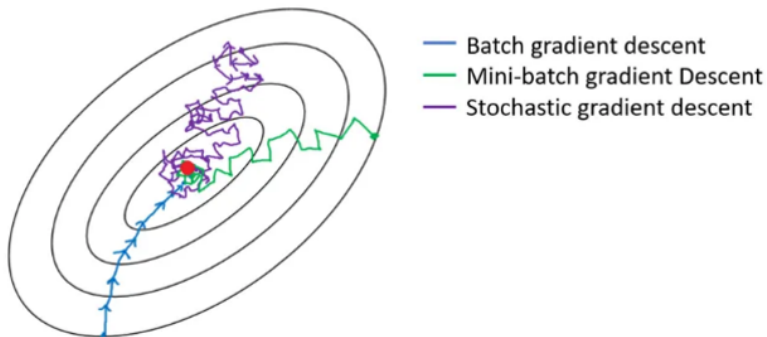
Algorithm 10.6: Mini-batch gradient decent

- 1 **Input:** initial parameter vector θ^0 , data $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$, learning rate η , batch size B , number of steps K
 - 2 **Output:** trained parameter vector $\theta^* = \theta^K$
 - 3 **for** $k=0, \dots, K-1$ **do**
 - 4 $\{\mathbf{x}^{(b)}, \mathbf{y}^{(b)}\}_{b=1}^B \sim \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$ $\triangleleft$ sample batch of training data
 - 5 $\tilde{\mathbf{g}} = \frac{1}{B} \sum_{b=1}^B \nabla_{\theta} \mathcal{L}(f_{\theta}(\mathbf{x}^{(b)}), \mathbf{y}^{(b)})$
 - 6 $\theta^k \leftarrow \theta^{k-1} - \eta \tilde{\mathbf{g}}$
-

Если размер батча $B = 1$, то это вариант стохастического градиентного спуска (SGD, stochastic gradient descent).

Различия GD, SGD, mbGD

Mini-batch gradient descent it will get some noise because of their



RMSProp (Root Mean Square Propagation): Adaptive Learning Rates

Параметры с большим градиентом будут делиться на большое число, поэтому будут иметь меньший шаг и наоборот.

$$\nabla L(\theta_t) = g_t^2 \quad (6)$$

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2 \quad (7)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \varepsilon} g_t \quad (8)$$

ADAM (Adaptive Moment Estimation)

momentum:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (9)$$

RMSProp:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (10)$$

bias correction ($\beta_i^t \rightarrow 0, t \rightarrow \infty$):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (11)$$

ADAM:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t \quad (12)$$